dev-flow

Manual do time

Fluxo de desenvolvimento com Claude Code: da demanda ao PR e à pipeline

O que você vai encontrar aqui

- O que é o dev-flow e que problema ele resolve
- O fluxo de ponta a ponta: demanda, funcional, técnica, plano, tarefas, PR e pipeline
- Os plugins, os agentes e o que cada um integra (git, Azure Boards, Azure Pipelines, ambiente local)
- Como instalar, configurar um projeto e usar no dia a dia
- As proteções automáticas e o que continua sendo decisão humana

Em uma frase: você explica a demanda em texto; o Claude Code escreve a especificação funcional e a técnica, monta o plano, implementa tarefa por tarefa (com testes, revisão, validação e commit) e prepara o PR, parando para a sua aprovação nos pontos que importam.

Versão dos plugins: 0.1.0 · Outubro/2026

1. O que é

O **dev-flow** é um *marketplace de plugins* do Claude Code. Cada plugin traz skills (comandos como `/fluxo:demanda`), agentes especializados e hooks (verificações automáticas que rodam antes de cada ação do Claude).

Ele nasceu da forma como o PBI 1001 (Acme.API.Ponte) foi desenvolvido: specs antes de código, tarefas pequenas, testes e validação real em cada tarefa, card atualizado com evidências e um commit por tarefa. O processo foi extraído para servir a qualquer projeto: back-end, front-end, scripts SQL ou vários repositórios na mesma demanda.

Por que plugin (e não MCP ou instruções soltas)

- **Processo é texto + scripts**: skills dizem ao modelo *como* trabalhar; scripts determinísticos fazem o que não pode depender de memória (fila de tarefas, chamadas de validação, bloqueios).
- **Instala uma vez, vale em todo projeto**: o que muda entre projetos fica num arquivo de perfil (`.claude/projeto.json`), não no plugin.
- **Acoplável**: instale só as stacks que você usa (.NET, SQL, front) e o Azure Boards só se o projeto usar.
- **Proteções reais**: o hook *guard* bloqueia de fato (push, segredo em commit, arquivo gerado, diretório proibido), em vez de depender do modelo lembrar.

Responsabilidades

O Claude faz	Você decide / faz
Escreve funcional, técnica e plano a partir do código real	Aprova (ou corrige) funcional, técnica e plano
Implementa, testa, revisa, valida e commita cada tarefa	Responde dúvidas de regra de negócio
Cria/atualiza cards no Boards (depois do seu "sim")	Faz push, abre/mergeia o PR
Gera o YAML de pipeline e diagnostica falhas	Aprova deploy em produção (no environment)

2. O fluxo de ponta a ponta

```
demanda (você explica em texto)
  → /fluxo:demanda  → 01-funcional.md                (*) você aprova
  → /fluxo:tecnica  → 02-tecnica.md (código real)     (*) você aprova
  → /fluxo:plano    → tarefas.json + 03-plano.md      (*) você aprova  (+ cards no Boards, opcional)
  → /fluxo:task todas → por tarefa: implementa → testa → revisor → validador → commit → feito
  → /fluxo:pr       → pr.md (+ PR aberto depois do seu push)
  → azure-pipelines → CI verde; deploy em produção com aprovação no environment

/fluxo:entregar faz tudo isso em sequência, parando só nos (*).
```

Etapa	Entrada	Saída	Portão
`/fluxo:demanda`	Seu texto (pedido, e-mail, print, conversa)	01-funcional.md: regras de negócio, requisitos, critérios de aceite (CA-n), fora de escopo, dúvidas	Você aprova
`/fluxo:tecnica`	Funcional aprovada + código dos repositórios afetados (agente analista)	02-tecnica.md: componentes, contratos, dados, integrações, segurança, testes por CA, riscos, implantação/CI	Você aprova
`/fluxo:plano`	Técnica aprovada	tarefas.json (fila) + 03-plano.md: tarefas pequenas, dependências, critérios e validação de cada uma	Você aprova
`/fluxo:task`	Próxima tarefa pronta da fila	Código + testes + commit; tarefa marcada como feita; card publicado (se Boards)	Para se houver dúvida
`/fluxo:pr`	Fila concluída	pr.md: o que muda, impacto, validação, pendências, implantação	Push/merge são seus
azure-pipelines	Stack do projeto	YAML de CI/CD por stack; deploy por ambiente	Produção com aprovação

Documentos de uma demanda

```
{docs}/{slug}/
  00-demanda.md     texto original do pedido
  01-funcional.md   RN, RF, critérios de aceite (CA-n), fora de escopo, dúvidas
  02-tecnica.md     componentes, contratos, dados, testes por CA, riscos, implantação
  03-plano.md       ordem, paralelismo, definição de pronto
  tarefas.json      fila (fonte da verdade): id, critérios, dependências, estado, wi, commit
  pr.md             descrição do PR
```

O `tarefas.json` é a fonte da verdade: estados `a_fazer`, `andamento`, `feito`, `bloqueada`. Uma tarefa está pronta quando todas as dependências estão feitas. Tarefas marcadas como protegidas nunca são executadas pelo Claude.

3. O ciclo de uma tarefa

É o coração do fluxo (`/fluxo:task`). Cada tarefa passa pelos mesmos passos, sempre:

#	Passo	O que acontece
0	Pré-condições	Valida o perfil, confere git limpo, gera o token de validação (se houver) e carrega as skills das stacks do projeto.
1	Escolher	Pega a próxima tarefa pronta da fila (ou a que você indicou), marca em andamento e move o card.
2	Implementar	Segue o padrão do código existente e as convenções da stack e do projeto.
3	Testes	Roda o comando de testes do perfil; teste novo para cada critério de aceite que couber.
4	Revisão	Agente **revisor** confere critérios de aceite, convenções e checklist da stack; achados graves são corrigidos antes de seguir.
5	Validação	Agente **validador** roda no ambiente real: sobe a API e chama as rotas, aplica e reverte SQL, roda um comando ou confere a tela.
6	Commit	Um commit por tarefa, no formato do perfil. O guard bloqueia commit com segredo.
7	Registro	Atualiza tarefas.json (commit, nota) e publica o card com Implementação, Testes e Validação.
8	Fechamento	Marca feito e resume o que foi feito, o que ficou pendente e o que depende de você.

Quando o Claude para e pergunta: regra de negócio ambígua, algo fora do escopo da demanda, falha de validação sem causa clara, alteração pendente que não é dele, ou qualquer situação listada em `pare_e_pergunte` no perfil do projeto.

Agentes

Agente	Quando entra	O que devolve
analista	Especificação técnica (um por repositório, em paralelo)	Onde e como os requisitos entram no código: arquivos-chave com linha, padrões, contratos, riscos. Somente leitura.
revisor	Passo 4 do ciclo	Achados objetivos com arquivo:linha, classificados por gravidade. Somente leitura.
validador	Passo 5 do ciclo	Tabela Chamada | Status | Resultado, pronta para o card.

4. Plugins e o que cada um integra

Plugin	O que traz	Integra com
fluxo (núcleo, obrigatório)	Skills iniciar, demanda, tecnica, plano, task, entregar, pr, status, daily, aprender; agentes analista, revisor, validador; hooks guard e contexto; scripts da fila, validação HTTP, subir/derrubar serviço e token; modelos de documento.	Claude Code, git, ambiente local (API, banco de dev, comandos)
azure-boards	Cria o PBI e as tasks a partir do plano, move estados, publica a descrição do card, importa um PBI existente.	Azure Boards (via MCP do Azure DevOps ou az boards)
azure-pipelines	Modelos YAML (dotnet, front, sql, deploy por ambiente); criar, ajustar e diagnosticar pipelines.	Azure Pipelines / environments
stack-dotnet	Convenções, armadilhas e checklist .NET; modelo de API em memória para testes de contrato.	dotnet build/test
stack-sql	Convenções SQL Server (idempotência, rollback pareado, lote); modelos de script e rollback.	Banco de desenvolvimento
stack-front	Convenções de front, verificação visual no navegador (desktop e mobile), checklist.	Navegador

Hooks (rodam sozinhos)

- **contexto** (início da sessão): mostra o projeto, a demanda ativa e a próxima tarefa pronta. Sem perfil, sugere `/fluxo:iniciar`.
- **guard** (antes de cada ação): aplica as proteções do perfil e bloqueia de verdade. Detalhes na seção 7.

O perfil do projeto (.claude/projeto.json)

É o contrato entre o processo genérico e o seu projeto. As skills não têm nada fixo: comando, porta, id de card, caminho, tudo vem daqui.

Seção	Para quê
`stacks, convencoes`	Quais skills de stack e quais arquivos de convenção valem
`docs`	Onde ficam as especificações e o tarefas.json
`rastreador`	Local ou Azure Boards (org, item raiz, protegidos, estados)
`comandos`	Build, teste, lint, verificação de pacotes vulneráveis
`servico, validacao`	Como subir a aplicação, token, URL, dados de teste, casos obrigatórios, máscaras de segredo
`git`	Branch base, formato do commit, coautor, push humano
`protecao`	Textos/diretórios proibidos, arquivos gerados, segredos (o guard aplica)
`pare_e_pergunte`	Situações do projeto em que o ciclo para

5. Instalação e primeiro uso

Pré-requisitos

- Claude Code (terminal, app desktop ou extensão de IDE) e Python 3 no PATH (os scripts usam só a biblioteca padrão).
- Acesso ao repositório do dev-flow.
- Para Boards: MCP do Azure DevOps configurado na máquina (seção 6).

Instalar (uma vez por máquina)

```
claude plugin marketplace add <caminho ou URL do repositório dev-flow>
claude plugin install fluxo@dev-flow
claude plugin install stack-dotnet@dev-flow     # conforme as stacks que você usa
claude plugin install stack-sql@dev-flow
claude plugin install stack-front@dev-flow
claude plugin install azure-boards@dev-flow     # se for usar o Boards
claude plugin install azure-pipelines@dev-flow  # se for gerar pipelines
```

Para conferir: `claude plugin list`. Os plugins valem a partir da próxima sessão do Claude Code.

Preparar um projeto (uma vez por projeto)

- Abra o Claude Code na pasta do projeto (pode estar vazia).
- Rode `/fluxo:iniciar`: detecta a stack e os comandos, cria `.claude/projeto.json`, um `CLAUDE.md` enxuto se não houver, a pasta de especificações, e faz `git init` se precisar.
- Revise o perfil: comandos de build/teste, como validar e o que é proibido mexer.

Usar numa demanda

Tudo de uma vez: `/fluxo:entregar` e explique a demanda em texto livre. **Passo a passo:**

Comando	Gera	Você
`/fluxo:demanda`	01-funcional.md	Responde as dúvidas e aprova
`/fluxo:tecnica`	02-tecnica.md	Aprova
`/fluxo:plano`	tarefas.json + 03-plano.md	Aprova
`/fluxo:task todas`	Código, testes e um commit por tarefa	Acompanha
`/fluxo:pr`	pr.md	Push e PR

No dia a dia

Comando	Para quê
`/fluxo:status`	Onde paramos, o que falta, bloqueios e divergências com o Boards
`/fluxo:task` / `/fluxo:task T3`	Só a próxima tarefa / uma tarefa específica
`/fluxo:daily`	Relatório da daily (ontem, situação, hoje, impedimentos)
`/fluxo:aprender`	Transforma uma correção ("isso não é assim") em regra durável no lugar certo

6. Azure Boards (opcional)

Sem configuração, tudo fica no `tarefas.json` e nada vai para o Azure. Para publicar cards como no PBI 1001:

1. Conectar o Azure DevOps (uma vez por máquina)

```
claude mcp add azure-devops -- cmd /c npx -y @azure-devops/mcp <org> -d core work work-items
```

2. Configurar o projeto (uma vez)

Preencha `rastreador` no perfil, ou peça "configura o Boards neste projeto, iteração X":

```
"rastreador": {
  "tipo": "azure-boards",
  "org": "https://dev.azure.com/<organização>",
  "projeto": "MeuProjeto",
  "area": "MeuProjeto\\Equipe",
  "iteracao": "MeuProjeto\\<sprint>",
  "responsavel": "<seu e-mail>",
  "item_raiz": null,
  "protegidos": [],
  "estados": { "a_fazer": "To Do", "andamento": "In Progress", "feito": "Done" }
}
```

- `item_raiz` vazio: o PBI é criado a partir da funcional.
- PBI que já existe: coloque o número. Se ele já tem tasks, peça "importa o PBI 2000" para trazê-las ao tarefas.json.

3. O que acontece no fluxo

Momento	No Boards
Plano aprovado	Mostra a lista de PBI e tasks e só cria depois do seu "sim"
Início de cada tarefa	Move a task para In Progress
Fim de cada tarefa	Publica a descrição do card (Implementação, Testes, Validação) e marca Done
`/fluxo:status`	Compara o Boards com o tarefas.json e aponta divergências

Pedidos avulsos: "cria os cards no Boards", "publica o card da T3", "move a task 2010 para In Progress".

Limites: só o PBI do perfil e as tasks filhas dele; itens em *protegidos* nunca são movidos nem editados; toda atualização confere a revisão do card antes de gravar, para não sobrescrever edição de outra pessoa.

7. Proteções automáticas

O hook **guard** roda antes de cada ação do Claude e bloqueia de fato (a ação não acontece), conforme o perfil:

Regra	Bloqueia
Push humano	Qualquer `git push` (o padrão quando há perfil)
Segredo em commit	Commit cujo diff tenha JWT, chave privada, chave AWS, token do GitHub ou padrões extras do perfil (sempre ativo)
Diretórios proibidos	Ler, editar ou buscar dentro deles (ex.: cópias antigas que não podem servir de referência)
Arquivos gerados	Editar à mão código gerado (SDKs, proxies), com exceções declaradas
Segredos	Ler arquivos de token/credencial ou citá-los em comandos, exceto pelos scripts autorizados
Textos proibidos	Ações cujo conteúdo case com expressões proibidas no perfil

Além disso: o validador mascara segredos nas saídas, o token nunca é impresso e certificados de ambiente de homologação são conferidos por impressão digital (sem desligar TLS).

8. Como o processo evolui

- **Correção de um projeto só** (comando, regra, dado de teste): vai no `projeto.json` ou no `CLAUDE.md` daquele projeto.
- **Correção que vale para todos** (passo do ciclo, modelo de documento, checklist de stack): vai no repositório dev-flow, com teste em tests/.
- Use `/fluxo:aprender` ao fim de uma demanda ou quando corrigir o Claude: ele decide o lugar certo e registra.
- Mudanças valem a partir da próxima sessão. Nova stack = um plugin pequeno (ver docs/novo-plugin-de-stack.md).

9. Perguntas frequentes

Preciso usar o fluxo inteiro?

Não. Cada skill funciona sozinha: dá para usar só `/fluxo:demanda` para escrever uma funcional, ou só `/fluxo:task` numa fila que já existe.

Funciona sem Azure?

Sim. Com `rastreador.tipo = local` tudo fica no tarefas.json; o Boards entra quando você quiser.

E projetos com vários repositórios?

A técnica roda um agente analista por repositório e o plano indica em qual repositório cada tarefa entra.

O Claude pode fazer push ou deploy?

Não. Push, merge e produção são sempre humanos; o guard bloqueia o push.

A sessão foi compactada / caiu. Perco o andamento?

Não. O estado está em arquivo (tarefas.json, documentos, commits). `/fluxo:status` retoma de onde parou.